



Home	Interview Questions	Java Programs	Pyramid Programs	Programming Challenges	Download Notes	What's new..?	
------	---------------------	---------------	------------------	------------------------	----------------	---------------	--

Collection Framework Based Questions- 1

1. What is the difference between `java.util.Iterator` and `java.util.ListIterator`?

`Iterator` - Enables you to traverse through a collection in the forward direction only, for obtaining or removing elements
`ListIterator` - extends `Iterator`, and allows bidirectional traversal of list and also allows the modification of elements.

2. What is the difference between the `HashMap` and a `Map`?

`Map` is an Interface which is part of Java collections framework. It stores data as key-value pairs.
`HashMap` is class that implements that using hashing technique.

3. What does `synchronized` means in `Hashtable` context?

`Synchronized` means only one thread can modify a hash table at one point of time. Any thread before performing an update on a hashtable will have to acquire a lock on the object while others will wait for lock to be released.

This adds an overhead which makes the `Hashtable` slower when compared to the `HashMap`. You must choose a `HashMap` when you want faster performance and choose the `Hashtable` when you want thread safety

4. Can we make a `HashMap` thread safe? If so, how?

`HashMap` can be made thread safe by synchronizing it. Example:
`Map m = Collections.synchronizedMap(hashMap);`

5. Why should I always use `ArrayList` over `Vector`?

You should choose an `ArrayList` over a `Vector` because - not all systems are multi-threaded and thread safety is not a mandatory requirement. In such cases, using a `Vector` is not only overkill but also adds a lot of overhead which might slow down system performance. Even if you are just iterating through a vector to display its contents, the lock on it still needs to be obtained which makes it much slower. So, choose the `ArrayList` over a `Vector` unless your system is multi-threaded.

6. What is an enumeration?

An enumeration is an interface containing methods for accessing the underlying data structure from which the enumeration is obtained. It is a construct which collection classes return when you request a collection of all the objects stored in the collection. It allows sequential access to all the elements stored in the collection.

7. What is the difference between `Enumeration` and `Iterator`?

The functionality of `Enumeration` interface is duplicated by the `Iterator` interface. `Iterator` has a `remove()` method while `Enumeration` doesn't. `Enumeration` acts as Read-only interface, because it has the methods only to traverse and fetch the objects, whereas using `Iterator` we can manipulate the objects also like adding and removing the objects. So `Enumeration` is used

Like 12k

G+1

2.2k

3 Online Now



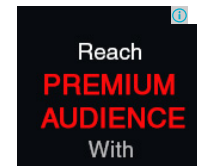
[meSleep Butterflies Laptop Skin](#)

(67)

Rs. 189.00 (details + delivery)

Search Program

Follow by Email



COLOMBIA

The Smart Marketing Platform for Content Promotion

SIGN UP NOW



Labels

Addition of number Advance Java Android AP Series Armstrong
 Number Arrays assertion Binary Addition binary Multiplication
 Binary to Decimal Binary to Hexadecimal C C++ Class Coding
 Standard Collection Conversion Core Java Daemon
 Thread Datatypes Decimal To Binary Decimal to hexadecimal
 Decimal to octal Diamond Durga Sir Durgasoft Encapsulation
 Exception Handling Factorial Features Fibonacci Garbage
 Collection Generic Root GP Series HCF and LCF Hibernnet HP
 Series HR Questions BM DB-2 IBPS Identifiers Indore interface
[Interview](#) [Interview](#) [Question](#) [Java](#)

whenever we want to make Collection objects as Read-only.

8. What is the difference between Sorting performance of Arrays.sort() vs Collections.sort()? Which one is faster? Which one to use and when?

Actually speaking, the underlying sorting algorithm & mechanism is the same in both cases. The only difference is type of input passed to them. Collections.sort() has a input as List so it does a conversion of the List to an array and vice versa which is an additional step while sorting. So this should be used when you are trying to sort a list. Arrays.sort is for arrays so the sorting is done directly on the array. So clearly it should be used when you have an array available with you and you want to sort it.

9. What is java.util.concurrent BlockingQueue? How it can be used?

The BlockingQueue is available since Java 1.5. Blocking Queue interface extends collection interface, which provides you the power of collections inside a queue. Blocking Queue is a type of Queue that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element. A typical usage example would be based on a producer-consumer scenario. Note that a BlockingQueue can safely be used with multiple producers and multiple consumers.

10. What is the ArrayBlockingQueue?

Ans- ArrayBlockingQueue is a implementation of blocking queue with an array used to store the queued objects. The head of the queue is that element that has been on the queue the longest time. The tail of the queue is that element that has been on the queue the shortest time. New elements are inserted at the tail of the queue, and the queue retrieval operations obtain elements at the head of the queue. ArrayBlockingQueue requires you to specify the capacity of queue at the object construction time itself. Once created, the capacity cannot be increased. This is a classic "bounded buffer" (fixed size buffer), in which a fixed-sized array holds elements inserted by producers and extracted by consumers. Attempts to put an element to a full queue will result in the put operation blocking; attempts to retrieve an element from an empty queue will be blocked.

11. Why doesn't Map interface extend Collection?

Though the Map interface is part of collections framework, it does not extend collection interface. This was done as a design decision by Sun when the Collection framework was created. Think of it this way - A Map contains key-value pair data while a general collection contains only a bunch of values. If the map were to extend the collection class, what will we do with the keys and how will we link the values to their corresponding keys? Alternately, if the collection were to extend the Map, then, where will we go to get the key information for all the values that are already in the Collection?

Get the idea? For all practically purposes a Map is considered a collection because it contains a bunch of data in key-value pairs but it does not explicitly extend the collection interface.

12. Which implementation of the List interface (ArrayList or LinkedList) provides for the fastest insertion of a new element into the list?

LinkedList.

Java Program Java4732 JavaEE JavaSE
JDBC JSP Linux Literals Matrix Multi threading Natraj sir
Natraj Technology NCR Notes Number Pattern Number
System Object Octal to binary Octal to Decimal OOPS Oracle
Palindrome Pattern PDF Polymorphism Practice Programs
Prime Number Programming Challenges
Programs Pyramid Real-Life-Example Rectangle
Pattern Recursion Reserved Words Reverse Number SAP SCJP
Serialization Series Series Servlet Session Silver Light
Software Companies Spring Star Pattern String Struts-2 Sum of
digits Swapping Two Array Top 10 triangle tutorial var-args
Method Video Tutorial Web Logic Web Service Written
Test

If you are surprised about the answer you see above, read the question again "Fastest Insertion of a new element into the list". The question is about inserts and not reading data. So, the linkedlist is faster. Because, when an element is inserted into the middle of the list in an ArrayList, the elements that follow the insertion point must be shifted to make room for the new element. The LinkedList is implemented using a doubly linked list; an insertion requires only the updating of the links at the point of insertion. Therefore, the LinkedList allows for fast insertions and deletions.

13. Which implementation of the List Interface (ArrayList or LinkedList) provides for faster reads during random access of data?

ArrayList implements the RandomAccess interface, and LinkedList does not. The commonly used ArrayList implementation uses primitive Object array for internal storage. Therefore an ArrayList is much faster than a LinkedList for random access, that is, when accessing arbitrary list elements using the get method. Note that the get method is implemented for LinkedLists, but it requires a sequential scan from the front or back of the list. This scan is very slow. For a LinkedList, there's no fast way to access the Nth element of the list.

14. Which implementation of the List Interface (ArrayList or LinkedList) uses more memory space for the same amount (number) of data?

Each element of a linked list (especially a doubly linked list) uses a bit more memory than its equivalent in array list, due to the need for next and previous pointers. So, the LinkedList uses more memory space when compared to an ArrayList of the same size

15. Where will you use ArrayList and Where will you use LinkedList?

If you frequently add elements to the beginning of the List or iterate over the List to delete elements from its interior, you should consider using LinkedList. These operations require constant-time in a LinkedList and linear-time (depending on the location of the object being added/removed) in an ArrayList.

If the purpose of your collection is to populate it once and read it multiple times from random locations in the list, then the ArrayList would be faster and a better choice because Positional access requires linear-time (depending on the location of the object being accessed) in a LinkedList and constant-time in an ArrayList.

In short - If you need to support random access, without inserting or removing elements from any place other than the end, then ArrayList offers the optimal collection. If, however, you need to frequently add and remove elements from the middle of the list and only access the list elements sequentially, then LinkedList offers the better implementation.

16. Why insertion and deletion in ArrayList is slow compared to LinkedList?

ArrayList internally uses an array to store the elements, when that array gets filled by inserting elements a new array of roughly 1.5 times the size of the original array is created and all the data of old array is copied to new array.

During deletion, all elements present in the array after the

deleted elements have to be moved one step back to fill the space created by deletion. In linked list data is stored in nodes that have reference to the previous node and the next node so adding element is simple as creating the node and updating the next pointer on the last node and the previous pointer on the new node. Deletion in linked list is fast because it involves only updating the next pointer in the node before the deleted node and updating the previous pointer in the node after the deleted node.

17.How do you traverse through a collection using its Iterator?

To use an iterator to traverse through the contents of a collection, follow these steps:

1. Obtain an iterator to the start of the collection by calling the collection's `iterator()` method.
2. Set up a loop that makes a call to `hasNext()`. Have the loop iterate as long as `hasNext()` returns true.
3. Within the loop, obtain each element by calling `next()`.

18.How do you remove elements during Iteration?

Iterator also has a method `remove()` when `remove` is called, the current element in the iteration is deleted.

19.What are the main implementations of the List interface?

The main implementations of the List interface are as follows :
ArrayList : Resizable-array implementation of the List interface.
The best all-around implementation of the List interface.
Vector : Synchronized resizable-array implementation of the List interface with additional "legacy methods."
LinkedList : Doubly-linked list implementation of the List interface. May provide better performance than the ArrayList implementation if elements are frequently inserted or deleted within the list. Useful for queues and double-ended queues (deques).

20.What are the advantages of ArrayList over arrays?

Some of the advantages ArrayList has over arrays are:

It can grow dynamically

It provides more powerful insertion and search mechanisms than arrays.

21.How to obtain Array from an ArrayList?

Array can be obtained from an ArrayList using `toArray()` method on ArrayList.

```
List arrayList = new ArrayList();  
arrayList.add(obj1);  
arrayList.add(obj2);  
arrayList.add(obj3);  
...  
Object a[] = arrayList.toArray();
```

22.Why are Iterators returned by ArrayList called Fail Fast?

Because, if list is structurally modified at any time after the iterator is created, in any way except through the iterator's own `remove` or `add` methods, the iterator will throw a `ConcurrentModificationException`. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future. So, it is considered fail fast.

23.How do you sort an ArrayList (or any list) of user-defined objects?

Create an implementation of the `java.lang.Comparable` interface that knows how to order your objects and pass it to

```
java.util.Collections.sort(List, Comparator).
```

24. What is the Comparable interface?

The Comparable interface is used to sort collections and arrays of objects using the `Collections.sort()` and `java.util.Arrays.sort()` methods respectively. The objects of the class implementing the Comparable interface can be ordered. The Comparable interface in the generic form is written as follows:

```
interface Comparable  
where T is the name of the type parameter.
```

All classes implementing the Comparable interface must implement the `compareTo()` method that has the return type as an integer. The signature of the `compareTo()` method is as follows:

```
int i = object1.compareTo(object2)
```

If `object1 < object2`: The value of `i` returned will be negative.

If `object1 > object2`: The value of `i` returned will be positive.

If `object1 = object2`: The value of `i` returned will be zero.

25. What are the differences between the Comparable and Comparator interfaces?

The Comparable uses the `compareTo()` method while the Comparator uses the `compare()` method.

You need to modify the class whose instance is going to be sorted and add code corresponding to the Comparable implementation.

Whereas, for the Comparator, you can have a separate class that has the sort logic/code.

In case of Comparable, Only one sort sequence can be created while Comparator can have multiple sort sequences.

 Like 2  Share  G+1  0

Posted by [Umesh Kushwaha](#)

 Recommend this on Google

Labels: [Collection](#), [Interview Question](#), [Java](#)

No comments yet



Add a comment as Vijay Kumar

[Newer Post](#)

[Home](#)

[Older Post](#)

Subscribe to: [Post Comments \(Atom\)](#)



Copyright © 2016. Simple template. Template images by [PLAINVIEW](#). Powered by [Blogger](#).